

HSE DASHBOARD PROJECT PLAN

Project Overview

Build a simple dynamic HSE Dashboard using:

Backend:

- Python
- FastAPI
- PostgreSQL

Frontend:

- HTML
- CSS
- JavaScript
- Bootstrap (optional)

The screenshots are only for reference.

The main goal is:

- Create records
- Store data in PostgreSQL
- Display live data in Dashboard
- Dashboard updates automatically from database

PROJECT MODULES

We will build only 5 modules:

1. Authentication & Users
2. Incidents
3. Tasks
4. Training
5. Dashboard

These modules are enough to build the dashboard shown in the screenshots.

MODULE 1 - AUTHENTICATION & USERS

Purpose: Manage users and login.

Features:

- User Login
- User Logout
- User List
- Add User
- Edit User
- Delete User

Database Table:

users

Columns:

- id
- name
- email
- password
- role
- created_at

APIs:

POST /login

GET /users

POST /users

PUT /users/{id}

DELETE /users/{id}

Frontend Pages:

- Login Page
- Users Page

Dashboard Usage:

- Identify logged-in user

- Show My Tasks
-

MODULE 2 - INCIDENTS

Purpose: Manage HSE incidents.

Features:

- Create Incident
- Edit Incident
- View Incident
- Delete Incident
- Change Status

Status:

- Open
- Investigation
- Resolved
- Closed

Database Table:

incidents

Columns:

- id
- title
- description
- category
- severity
- status
- created_at

APIs:

GET /incidents

POST /incidents

PUT /incidents/{id}

DELETE /incidents/{id}

Frontend Pages:

- Incident List
- Create Incident
- Edit Incident

Dashboard Usage:

- Total Incidents
 - Open Incidents
 - Investigations
 - Incident Trend Chart
 - Days Since Last Incident
-

MODULE 3 - TASKS

Purpose: Manage department tasks.

Features:

- Create Task
- Edit Task
- Assign Task
- Update Status
- Delete Task

Status:

- Todo
- In Progress
- Completed

Database Table:

tasks

Columns:

- id
- title
- description
- assigned_to
- status
- due_date

APIs:

GET /tasks

POST /tasks

PUT /tasks/{id}

DELETE /tasks/{id}

Frontend Pages:

- Task List
- Create Task
- Edit Task

Dashboard Usage:

- Active Tasks
 - My Tasks
 - Completed Tasks
-

MODULE 4 - TRAINING

Purpose: Manage employee training.

Features:

- Create Training
- Assign Training
- Mark Complete
- View Training

Database Table:

training

Columns:

- id
- title
- assigned_to
- status
- due_date

Status:

- Pending
- Completed
- Overdue

APIs:

GET /training

POST /training

PUT /training/{id}

DELETE /training/{id}

Frontend Pages:

- Training List
- Create Training
- Edit Training

Dashboard Usage:

- Training Completed
- Training Pending
- Training Overdue

MODULE 5 - DASHBOARD

Purpose: Display live statistics.

Dashboard Cards:

1. Days Since Last Incident
2. Active Department Tasks
3. My Tasks
4. Training Completed
5. Training Pending

6. Training Overdue

7. HSE Incidents

8. HSE Investigations

Charts:

- Monthly Incident Trend

Dashboard APIs:

GET /dashboard/summary

GET /dashboard/incidents-chart

Example Response:

```
{ "days_since_last_incident": 5, "active_tasks": 12, "my_tasks": 4, "training_completed": 20,
"training_pending": 10, "training_overdue": 2, "incidents": 6, "investigations": 2 }
```

TEAM MEMBER ASSIGNMENT

=====

MEMBER 1 AUTHENTICATION & USERS

=====

Tasks:

Backend:

- Create User Model
- Create User APIs
- Implement Login
- Implement JWT Authentication

Database:

- Create users table

Frontend:

- Login Page

- Users Page

Deliverables:

- Working Login
- User CRUD

=====

MEMBER 2 INCIDENTS MODULE

=====

Tasks:

Backend:

- Incident Model
- Incident APIs
- Incident Statistics API

Database:

- Create incidents table

Frontend:

- Incident List
- Create Incident Form
- Edit Incident Form

Deliverables:

- Incident CRUD
- Incident Statistics

=====

MEMBER 3 TASKS MODULE

=====

Tasks:

Backend:

- Task Model
- Task APIs

Database:

- Create tasks table

Frontend:

- Task List
- Create Task Form
- Edit Task Form

Deliverables:

- Task CRUD

=====

MEMBER 4 TRAINING MODULE

=====

Tasks:

Backend:

- Training Model
- Training APIs

Database:

- Create training table

Frontend:

- Training List
- Create Training Form
- Edit Training Form

Deliverables:

- Training CRUD

=====

MEMBER 5 DASHBOARD MODULE

=====

Tasks:

Backend:

- Dashboard Summary API
- Dashboard Chart API

Frontend:

- Dashboard Page
- KPI Cards
- Incident Chart

Dashboard Cards:

- Incidents
- Investigations
- Tasks
- Training

Deliverables:

- Complete Dashboard

=====

MEMBER 6 SETUP & INTEGRATION

=====

Tasks:

Backend Setup:

- FastAPI Project Setup
- PostgreSQL Connection
- SQLAlchemy Setup
- Alembic Setup

Frontend Setup:

- HTML Template
- Bootstrap Setup
- Sidebar
- Header
- Navigation Menu

Database:

- Database Creation

- Initial Migrations

Integration:

- Connect Frontend with APIs
- Test All Modules

Deliverables:

- Complete Project Structure
 - Integration Support
-

SIMPLE DATABASE TABLES

users

incidents

tasks

training

Only 4 main business tables.

Keep it simple.

PROJECT FOLDER STRUCTURE

backend/

app/

auth/

users/

incidents/

tasks/

training/

dashboard/

models/

database/

main.py

frontend/

pages/

css/

js/

dashboard.html

users.html

incidents.html

tasks.html

training.html

login.html

DEVELOPMENT ROADMAP

WEEK 1

Member 6

- Setup FastAPI
- Setup PostgreSQL
- Setup Frontend Template

Member 1

- Users Module

Member 2

- Incidents Module

Member 3

- Tasks Module

Member 4

- Training Module

Member 5

- Dashboard Design

Goal:

All CRUD APIs should work.

WEEK 2

Members 1-4

- Finish Forms
- Finish APIs
- Add Validation

Member 5

- Connect Dashboard APIs

Member 6

- Integration Support

Goal:

All modules connected to database.

WEEK 3

Team Integration

Tasks:

- Connect Dashboard
- Fix Bugs
- Test APIs
- Test Database
- Test Frontend

Goal:

When records are added, dashboard updates automatically.

FINAL SIDEBAR

Dashboard

Incidents

Tasks

Training

Users

Settings

SUCCESS CRITERIA

- ✓ User can login
- ✓ User can create incidents
- ✓ User can create tasks
- ✓ User can create training records
- ✓ Data saved in PostgreSQL
- ✓ Dashboard reads data from database
- ✓ Dashboard statistics update automatically
- ✓ Dashboard shows charts

✓ Simple and clean UI

✓ Working end-to-end project